Part B: Sharing One Fronthaul Link Well

Deadline-feasible, uncertainty-aware budget allocation across O-RAN 7-2x cells: the KKT newsvendor rule, a Bayesian Transformer forecaster, and the back-pressure newsvendor

Mayank Yadav Harshvardhan Choudhary
B.Tech. Project — Final Evaluation, November 2026

Part B at a glance

Three steps, each building on the previous one

B1 KKT newsvendor rule

Forecast r* as a range (GBM)
Split the link at equal weighted risk
Best of 12 controllers, 2 families
NCC 2027 submission

Extended study

MDP view + Markov-fluid theory
3rd traffic family (Markov)
CEM policy & predictive MaxWeight
beat the KKT rule in 9/10

B2 Bayesian Transformer

LCQT: patch + cross-cell attention
Bayesian mixture, differentiable KKT
Best KKT variant, small gain
→ the gap is structural

B3 Back-pressure newsvendor

Back-pressure base + KKT margins
Contains both strongest baselines
Beats them in 10/10 scenarios
of 3 traffic families

Keep the structure, learn only what is uncertain — then fix the structure where it is wrong.

The problem: one link, eight cells, decide in advance

- 8 cells: 3 low-latency (LL, deadline 2 ms, weight 3) + 5 eMBB (10 ms, weight 1)
- One 25GE link, 3 % for control → C = 12.125 Mbit per 0.5 ms slot
- Aggregate cell peak = 2.44 × C (statistical multiplexing)
- Every T = 10 ms: per-cell budgets r_i, held for the interval
- Telemetry is τ = 1 ms old; the future is never seen
- Bits waiting longer than their deadline are dropped = violations

Why decide in advance?

- Near-RT RIC / E2 loop ≥ 10 ms
- Switch shaping not changed per slot
- DU schedulers cannot coordinate inside the sub-ms pipeline

⇒ commit for T + τ without seeing traffic

Question

Given a forecast range of each cell's need, how should the link be split?

Someone must lose data when bursts coincide; the controller decides who.

System model

Fronthaul load follows scheduled resource blocks, not user bits

Cost of one PRB-layer per slot (Part A enters here)

$\beta(b) = (12 \cdot 2 \cdot b + 8) \cdot 14 \cdot 1.08$

$\beta(9) = 3{,}387$ bits; BFP-6: 0.68×, BFP-12: 1.32×, BFP-14: 1.54×

Cell peak = 273 PRBs × 4 layers × $\beta \approx 7.4$ Gbit/s

Fronthaul demand and decision

$d_i(t) = \lceil a_i(t) / \eta_i(t) \rceil \cdot \beta$ (user bits ÷ spectral eff.)

$\Sigma_i r_i \le C, \quad 0 \le r_i \le cap_i$

FIFO DU queue per cell; drop after D_i slots

Primary metric (fixed before tuning)

Priority-weighted violation ratio

$V_w = \Sigma\, w_i \cdot violated_i / \Sigma\, w_i \cdot arrived_i$

= (3·LL dropped + eMBB dropped) / (3·LL arrived + eMBB arrived)

Lower is better.

Also reported: per-class violations, unweighted ratio, utilisation, delay, Jain fairness, decision time.

Two cells with the same user traffic can differ 2–3× in fronthaul load.

B1: The KKT Newsvendor Rule

Forecast the right quantity, as a range, and split at equal risk

Key idea 1: forecast the deadline-feasible rate r*

Smallest constant rate at which no bit misses its deadline

$r^*(a, D) = \max$ over windows $[s, e]$ of $\Sigma_{t=s..e}\, a_t \;/\; (e - s + 1 + D)$ (network-calculus service curve; tight in tests)

- Long deadline → r* ≈ mean rate
- Short deadline → r* ≈ peak burst rate
- Example: burst of 8, mean 1
  - D = 1 slot → r* = 4; D = 7 slots → r* = 1
- 2 ms class: r* is 2–3× the mean and right-skewed

Demand of every deadline-aware controller

need_i = B_i + r*_i

B_i = backlog_i / slack_i (what is already queued)

r*_i = what has not arrived yet

Controllers differ only in how they estimate r*_i and split a shortage.

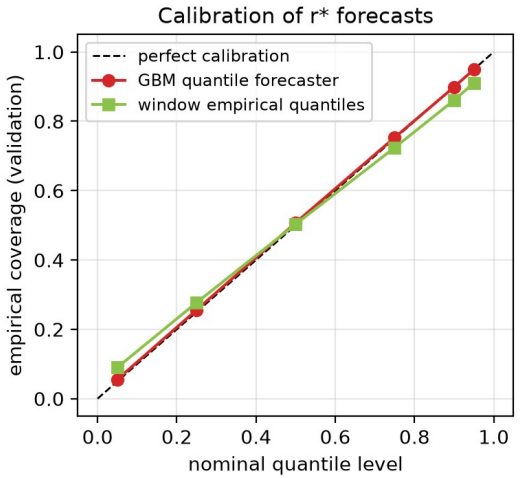
A single number is a poor basis for reserving a skewed quantity ⇒ forecast a range.

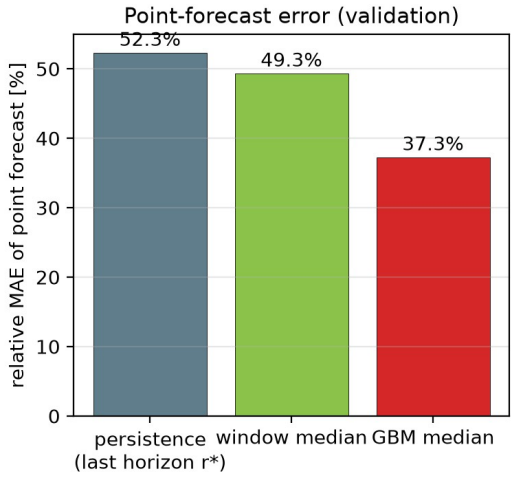
Key idea 2: a quantile forecaster

Six gradient-boosted tree models, one per quantile level

- Target: y_i = r*(next horizon) / cell peak
- 28 features, past only: horizon means, EWMAs, last 4 slots, r* of last 3 horizons, spectral eff., D/H, cell id
- 6 models (α = 5/25/50/75/90/95 %), 300 trees, 15 leaves
- 101k rows, 12 s training on CPU
- Seeds: train 1000–1015, val 2000–2003, test 3000–3019

Validation	GBM	Baseline
Coverage at 5 / 50 / 95 %	5.6 / 50.7 / 94.9	—
Median rel. error	37 %	52 % (persist.)
Pinball loss	0.0224	0.0293 (window)





Well calibrated, cheap and CPU-only.

Key idea 3: the KKT (constrained newsvendor) rule

Objective for one interval (forecast distribution F_i of r^*_i)

$\max_r \ \Sigma_i w_i \cdot E[\min(B_i + r^*_i, r_i)] \quad \text{s.t.} \ \Sigma_i r_i \le C, \ 0 \le r_i \le cap_i$

Concave; slope = $w_i \cdot P$(the next bit of budget is needed)

Closed-form solution

$r_i(\lambda) = \text{clip}(B_i + F_i^{-1}(1 - \lambda / w_i), 0, cap_i)$

- one price λ for all cells, 60-step bisection
- $\lambda = 0$ if the link is not full
- tens of µs for 8 cells

Toy example ($C = 10$, $\lambda = 0.6$)

LL ($w = 3$): level $1 - 0.6/3 = 0.8 \rightarrow 80$ % quantile
eMBB ($w = 1$): level $1 - 0.6 = 0.4 \rightarrow 40$ % quantile

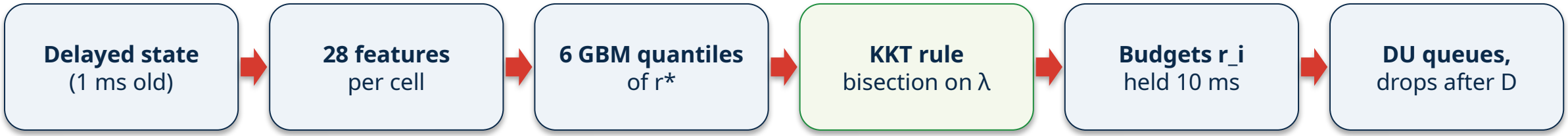
Wide forecast or high priority → more margin;
confident forecast → about its median.

Every cell is cut at the same weighted risk: margins are derived, not tuned.

Properties and the control loop

Proposition	In plain words
1 Feasibility	Whatever the forecast, budgets fit the caps and $\Sigma r_i \le C$: the link is never overloaded
2 Equal weighted risk	$w_i (1 - F_i(r_i - B_i)) = \lambda$ for every cell; a point forecast reduces it to a priority fill
3 Backlog first	Queued bits are funded before uncertain demand of equal or lower weight
4 Miscalibration	Forecast off by ε (Kolmogorov) $\Rightarrow$ loss $\le 2 \varepsilon w_{max} C$: linear, so it degrades gently

Control loop (every 10 ms)



Properties of one interval — no closed-loop optimality claim.

Baselines and evaluation protocol

All baselines tuned on validation seeds; identical traces for every controller

Controller	Estimate of r*	Split
Static equal	none	C / N
Reactive proportional	EWMA, no deadlines	proportional
MaxWeight	priority × queue	max-weight
Deadline-aware reactive	last observed r*	water-fill
Point forecast	GBM median	water-fill
Fixed global quantile	GBM, level 0.75	water-fill
Proposed (window)	last 30 intervals, no ML	KKT
Proposed	**GBM quantiles**	**KKT**
Oracle / Oracle + KKT	true r*	WF / KKT

Family S (synthetic, bursty)

Gamma + Pareto ON/OFF bursts
low 0.50, moderate 0.65, high 0.80,
flash crowds, held-out shift

Family G (3GPP only)

TR 38.901 UMa, TS 38.214 CQI
FTP Model 3 (eMBB), XR video (LL)
Demand = PRB scheduler output

20 test seeds × 10 s, paired bootstrap CIs, Holm-corrected Wilcoxon tests.

B1 result, Family S: V_w [%] (mean of 20 test seeds)

Method	low	mod.	high	flash	shift
Static equal	4.45	9.65	15.46	19.15	22.31
Reactive proportional	0.83	4.11	9.20	12.27	11.70
MaxWeight	0.90	4.85	11.54	15.24	12.31
Deadline-aware reactive	0.52	4.24	11.03	14.51	12.56
Point forecast	0.27	3.23	9.70	13.33	11.30
Fixed global quantile	0.34	4.35	11.76	15.09	12.85
Window quantiles (no ML)	0.25	2.57	8.72	12.67	10.22
Proposed (GBM + KKT)	**0.15**	**2.15**	**8.00**	**11.76**	**10.04**
Oracle water-fill	0.07	1.62	6.89	10.16	6.69

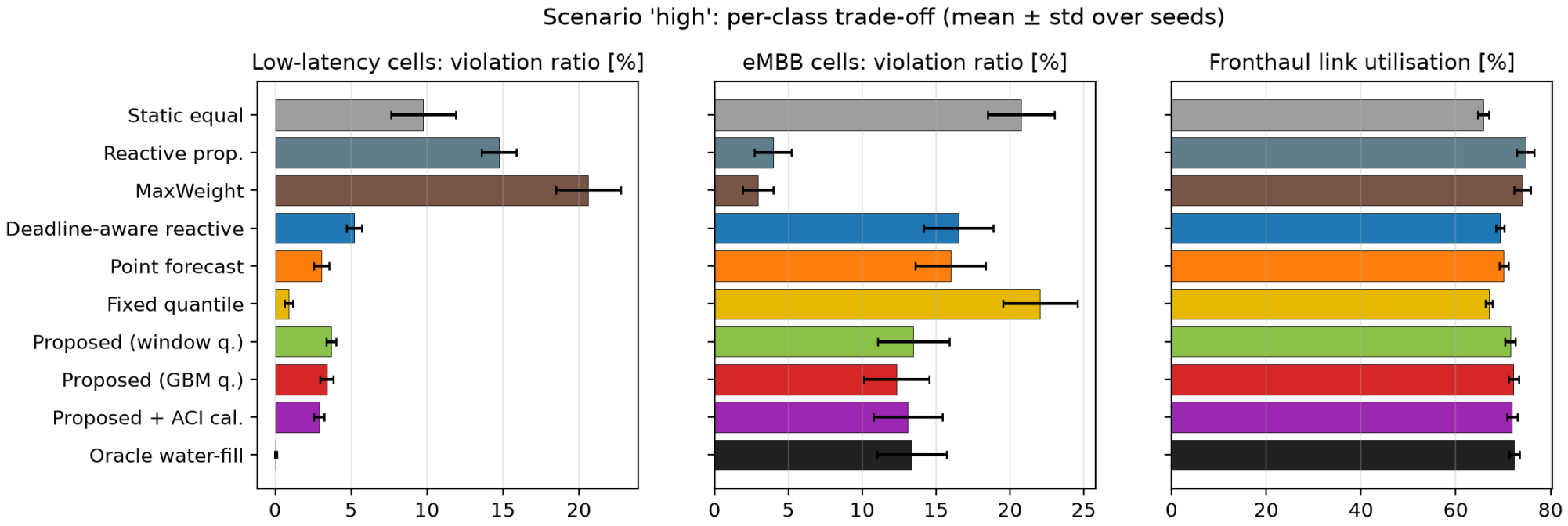
Paired vs proposed

vs point forecast: −11 to −43 %
vs fixed quantile: −22 to −54 %
vs deadline-aware: −19 to −70 %
vs MaxWeight: −18 to −83 %
all 20/20 seeds, $p < 0.01$

Weak: vs reactive prop. under flash crowds only −4 %
(16/20, $p = 0.03$)

Best implementable controller in all five scenarios.

Where the gain comes from (scenario high)



- Unaware controllers starve LL (14.7 %, 20.6 %); aware ones pay with eMBB
- Proposed: lowest eMBB loss (12.3 %) and highest utilisation (72.2 %) in the deadline-aware family

Margin only where the forecast is wide.

Ablations: what carries the gain?

Ablation (same everything else)	Effect of the proposed choice	Reading
Range vs single number (median)	−11 % to −43 %, 20/20 seeds	Representing uncertainty matters
Per-cell KKT level vs one global level	−22 % to −54 %	Level must be set per cell by the link
GBM vs window quantiles (no ML)	−7 % to −37 %; only −2 % under shift	Most of the gain is the rule
Online ACI calibration on/off	1–9 % worse; 10 % worse under shift	Per-stream correction does not transfer

Family G (3GPP) reproduces it

High load: 8.01 % — −11 % vs point forecast, −31 % vs deadline-aware, −50 % vs reactive prop., 20/20 seeds;
also best under the unweighted ratio. Oracle + KKT gap: 24–40 % of the remaining loss at high load is forecast error.

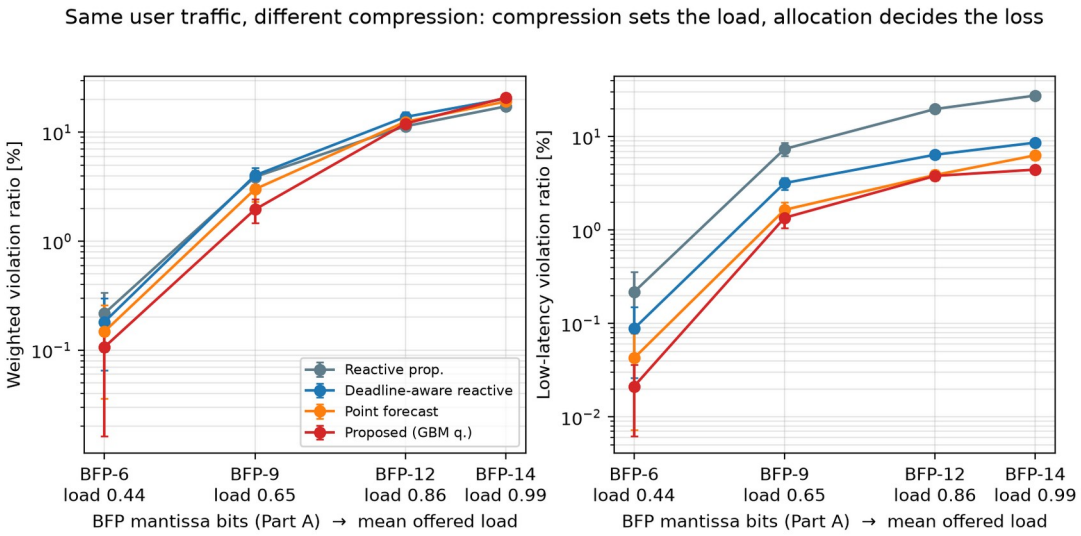
ML is useful, not necessary: the r* target and the KKT split do most of the work.

Honest caveats and operating regime

LL weight (S high)	1	2	3	5
Reactive prop.	6.55	8.13	9.20	10.56
Proposed	6.58	8.35	8.00	6.93

Interval T	2 ms	5 ms	10 ms	20 ms
Reactive prop.	3.91	7.42	8.91	11.80
Proposed	3.07	5.07	7.71	13.15

- Needs an LL weight ≥ 2–3 (priority trade)
- Loses at T = 20 ms; near tie at τ = 5 ms
- Sweeps use 10 seeds (hence 7.71 vs 8.00)



Part A × B: BFP 6/9/12/14 bits → load 0.44/0.65/0.86/0.99.
Allocation matters at 0.5–0.8; at ≈ 1.0 nothing helps.

Compression sets the regime; allocation decides the loss inside it.

Extended Study

Theory, a third traffic family, and stronger controllers

Extended study: stronger controllers exist

- Real problem: a POMDP — intractable
- Three approximation families:
  - one-step stochastic program (KKT rule)
  - Lyapunov drift: predictive MaxWeight
  - policy search: CEM, 3 parameters
- Family M: Markov-modulated traffic
- Theory: r* linked to effective bandwidth;
  - bounds valid but 25–100× loose

V_w [%]	KKT (GBM)	Pred. MW	CEM
S moderate	2.15	1.62	1.77
S high	8.00	6.15	6.57
S flash	11.76	9.69	10.20
G high	8.01	11.36	7.17
M high	4.94	4.79	3.93
M shift	5.52	4.80	4.17

λ diagnostic: link full ~100 % of intervals; λ* ≥ w_eMBB in 50–58 % → eMBB already at its lowest quantile; re-tuning the weight gains 1–4 %

CEM best in 6/10, predictive MaxWeight in 3/10: the gap is structural, not a constant.

B2: A Bayesian Transformer Forecaster

Can a better forecaster close the gap? (Family S)

LCQT: link-aware cross-cell quantile Transformer

A drop-in replacement for the GBM forecaster

Patch tokens
16 patches × 22 slots
+ r*, mean, max

Temporal encoder
2 layers, d = 48,
4 heads

FiLM
deadline D/H,
priority p

Cross-cell attn.
over the 8 cells
on the link

Non-crossing head
6 quantiles
$q_k = q_{k-1} +$ softplus

Bayesian mixture
3 seeds × 8 dropout
→ mixture $\hat{F}_i$

Differentiable KKT layer (DF-LCQT)

Forward: bisection for λ^*; backward: implicit gradient

$\partial\lambda^*/\partial q = -(\partial g/\partial q) / (\partial g/\partial \lambda)$, $g = \Sigma r_i - C$

Loss = $\Sigma\, p_i (D_{i+1})(B_i + r^*_i - r_i)^+ + \beta \cdot$ pinball

Contexts logged closed-loop, one DAgger round

	LCQT
Parameters	63 k (44 k without cross-cell)
Training	40 epochs, early stop on validation
Time	≈ 6 min, 4-core CPU
Data	same seeds and target as GBM

Each block is prior art; we found no paper that combines them for a shared link.

B2 forecast quality

Same validation rows for every model; shift = held-out test (out of distribution)

Model	Pinball	Pinball LL	90 % cov.	90 % width	Shift 90 % cov.	Shift LL q95 cov.
GBM (B1)	0.0224	0.0259	0.894	1.51	—	—
LCQT	0.0214	0.0251	0.902	1.36	0.795	0.855
LCQT + MC dropout (16)	0.0214	0.0251	0.911	1.41	0.802	0.870
Deep ensemble (3)	0.0212	0.0248	0.912	1.37	0.806	0.864
Ensemble × MC (3×8)	**0.0213**	**0.0249**	**0.919**	**1.40**	**0.808**	**0.873**
DF-LCQT (decision-focused)	0.0224	0.0273	0.907	1.38	0.830	0.822

- Transformer: pinball −4.5 %, 90 % interval −8 % at higher coverage than GBM
- Cross-cell layer and priority-weighted loss: no forecast gain (equal within noise)
- Bayesian layer improves calibration under shift (LL q95: 0.855 → 0.873) but cannot remove under-coverage

A better forecast — but only modestly better.

B2 closed loop: V_w [%] on Family S (20 test seeds)

Controller	low	mod.	high	flash	shift
GBM + KKT (B1)	0.15	2.15	8.00	11.76	10.04
LCQT median + water-fill	0.25	3.14	9.76	13.47	11.51
LCQT + KKT	0.13	2.05	7.91	11.81	10.06
Bayesian LCQT + KKT	**0.12**	**2.02**	**7.89**	**11.78**	**10.02**
DF-LCQT + KKT	0.18	2.26	7.93	11.62	10.15
CEM policy	0.13	1.77	6.57	10.20	9.73
Predictive MaxWeight	0.14	1.62	6.15	9.69	10.25
Oracle + KKT	0.05	0.88	4.77	8.10	5.84

Bayesian vs GBM	Change	Wins
low	−20.8 %	20/20
moderate	−6.1 %	20/20
high	−1.4 %	20/20
flash	+0.2 % (n.s.)	9/20
shift	−0.2 % (n.s.)	14/20

- Patch Transformer: the main gain
- Bayesian mixture: small, consistent
- Cross-cell, priority loss: no gain
- DF-LCQT: −1.2 % flash, +14 % low

Best KKT variant, but CEM / predictive MaxWeight stay 14–28 % better: the gap is structural.

B2: honest assessment

What improved

- Bayesian LCQT beats GBM + KKT on every seed at low–high load; never loses significantly
- Best calibration under shift
- LCQT alone is cheaper per decision than GBM (2.7 vs 3.5 ms)
- Differentiable KKT layer works and is unit-tested (matches numpy to 1e-6)

What did not

- Gains small at high load (−1.4 %)
- Decision-focused gain on validation did not transfer to test at low/moderate load
- Bayesian layer: 17–21 ms in Python (> the 10 ms interval) — needs compiling
- Family S only

Next: keep the forecaster, change the decision structure (B3).

B3: The Back-Pressure Newsvendor

One policy class that contains both strongest baselines

Why the KKT rule loses to back-pressure (S high)

Controller	V_w	LL viol.	eMBB viol.	Util.
KKT rule (GBM)	8.00	3.4	12.3	72.2
CEM class-level	6.57	6.0	7.1	74.7
Pred. MaxWeight (CEM-tuned)	5.98	6.0	5.9	75.5
BPN v1	**5.23**	**5.5**	**4.9**	**76.1**
BPN v2	**5.32**	**5.8**	**4.8**	**76.1**
Oracle + KKT	4.77	0.1	9.2	74.8

Diagnosis

A budget is a per-cell cap: reserved-but-unused capacity is lost.

KKT over-protects LL and starves eMBB.

Back-pressure wastes less but ignores deadlines and uncertainty.

Idea: fill the base in back-pressure order, then spend the leftover on newsvendor margins.

BPN: back-pressure base + KKT margins

8 parameters, learned by CEM on training seeds only

Stage 1 — base, in back-pressure order

$b_i = a \cdot Q_i/T + (1 - a) \cdot B_i$ (drain rate vs deadline backlog)

$d_i = b_i + (1 + \mu) \cdot \hat{F}_i^{-1}(p^0_class)$

Shortage $\propto v_class \cdot (\pi_i / \bar{\pi})^{\eta}$, $\pi_i = Q_i + \ell \cdot H \cdot median_i$

$\eta = 0$: class-weighted water-fill; $\eta = \eta_max$: strict back-pressure

Stage 2 — leftover by the KKT rule

$r_i = \max(r_i^{(1)}, b_i + \hat{F}_i^{-1}(1 - \lambda/\omega_class))$, $\Sigma r_i = C$

Contains predictive MaxWeight and the CEM class-level policy exactly (unit-tested).
Baselines re-tuned with the same CEM search: equal tuning effort.

Learned	η	a	(p^0_LL, p^0_eMBB)
S (bursty)	6 (greedy)	0.98	(0.41, 0.45)
G (3GPP)	0	0.31	(0.94, 0.73)
M (Markov)	0	0.34	(0.84, 0.31)

The search recovers the regime map

Short bursts (S) → back-pressure
Long bursts (FTP files, XR frames) → reservation

One policy class, both strongest baselines inside it: the search can only match or beat them.

B3 result, Family S: V_w [%] (20 test seeds)

Forecaster	Controller	low	moderate	high	flash	shift
GBM	KKT rule (B1)	0.15	2.15	8.00	11.76	10.04
GBM	Pred. MaxWeight (CEM-tuned)	0.16	1.66	5.98	9.36	10.46
GBM	CEM class-level (re-searched)	0.13	1.73	6.58	10.23	9.76
GBM	BPN v1 (6 param.)	0.10	1.21	5.23	8.58	9.23
GBM	BPN v2 (8 param.)	0.10	1.21	5.32	8.70	9.26
LCQT	BPN v2	0.08	1.16	5.25	8.59	9.24
Bayesian LCQT	BPN v1	0.08	1.11	5.05	8.34	8.92
Bayesian LCQT	**BPN v2**	**0.07**	**1.12**	**5.21**	**8.55**	**9.12**
—	Oracle + KKT (true r*)	0.05	0.88	4.77	8.10	5.84

Same forecaster: BPN beats both baselines everywhere; within 6–10 % of the oracle at high load.

B3 generalisation: Families G (3GPP) and M (Markov)

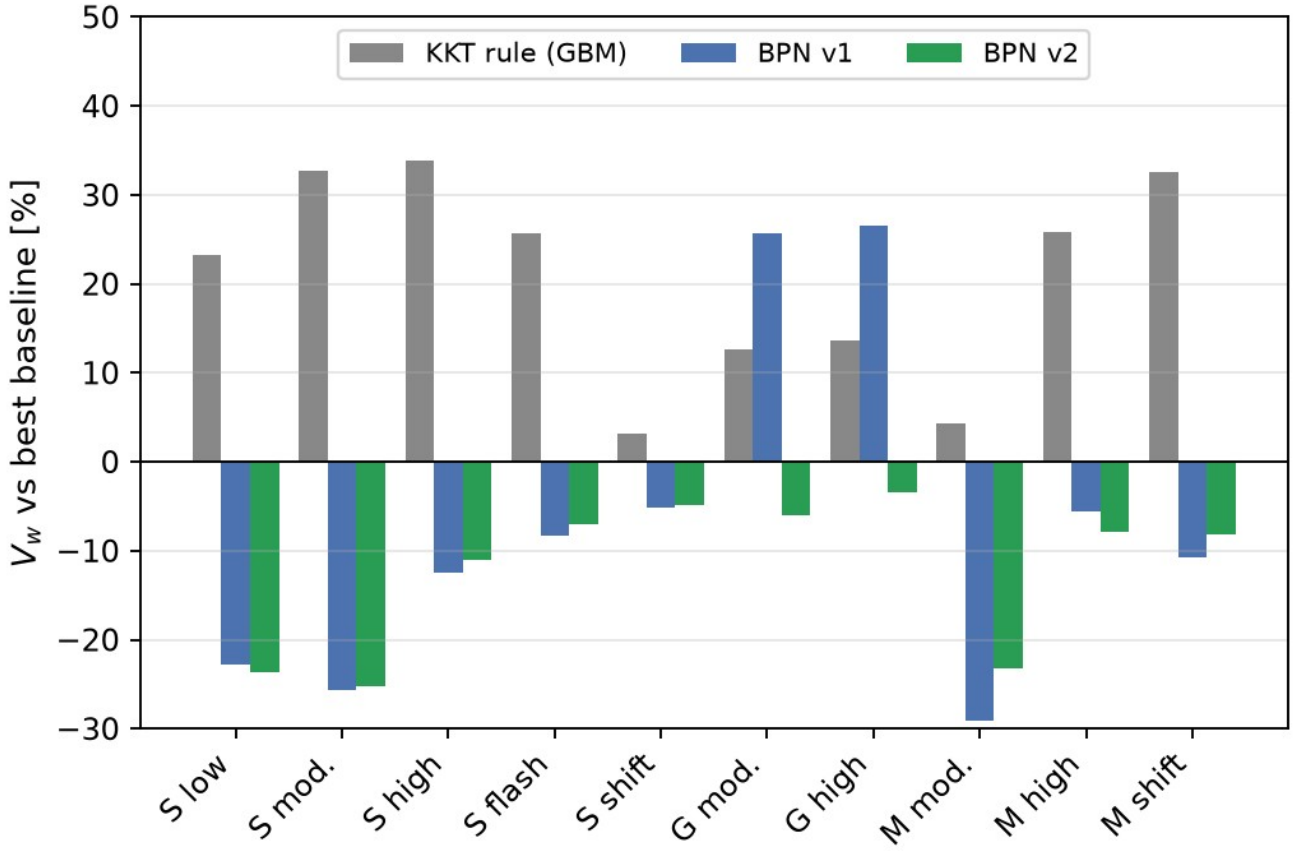
V_w [%], GBM forecaster

Controller	G moderate	G high	M moderate	M high	M shift
KKT rule (B1)	5.08	8.01	0.92	4.94	5.52
Pred. MaxWeight (CEM-tuned)	6.75	10.02	1.61	4.82	4.82
CEM class-level (re-searched)	4.51	7.05	0.94	3.94	4.22
BPN v1	5.67	8.92	0.63	3.71	3.72
BPN v2	**4.24**	**6.81**	**0.68**	**3.62**	**3.82**
Oracle + KKT	3.47	6.06	0.03	1.84	2.31

- BPN v1 lost on G: strict back-pressure cannot represent reservation, which wins on long FTP bursts
- v2 contains the reservation policy and fixes it
- Training cost (BPN v2 / best baseline): S 0.0387 / 0.0436, G 0.0527 / 0.0555, M 0.0196 / 0.0226

v2 is the version that generalises across all three families.

BPN v2 vs the best baseline in each scenario



Scenario	vs best baseline	Wins
S low	−23.7 %	15/20
S moderate	−25.3 %	20/20
S high	−11.0 %	20/20
S flash	−7.0 %	20/20
S shift	−4.8 %	20/20
G moderate	−5.9 %	20/20
G high	−3.4 %	19/20
M moderate	−23.3 %	20/20
M high	−7.9 %	20/20
M shift	−8.2 %	20/20

10/10 scenarios, 3 families, all $p \leq 0.024$ (Holm–Wilcoxon).

B3: what is now claimable, and what is still needed

Claimable

- Beats both strongest baselines, each in its best tuning, in 10/10 scenarios
- Equal search effort, training seeds only; baselines are special cases of the policy
- Learned η explains when to reserve and when to back-pressure: interpretable
- 2.2–3.5 ms per decision (Python) (Bayesian forecaster: $\approx$ 20 ms)

Still needed

- Real per-cell PRB / fronthaul traces
- Deep-RL xApp (PPO) baseline; Lyapunov drift-plus-penalty with deadline queues
- BPN over T, τ and load (only T = 10 ms, τ = 1 ms so far)
- Policy learned per family; transfer untested
- No optimality proof for the hybrid

A controller that beats the strongest baselines — with its limits stated.

Part B summary

Step	Headline result
B1 KKT newsvendor rule	Best of 12 controllers in all scenarios of 2 families: −11 to −43 % vs forecast median, every seed
Ablations	Most of the gain is the r* target + KKT split; ML useful, not necessary; ACI calibration hurt
Caveats	Needs LL weight ≥ 2–3; T ≤ a few × the shortest deadline; allocation matters at load 0.5–0.8
Extended study	r* linked to effective bandwidth; CEM and pred. MaxWeight beat the KKT rule in 9/10: structural gap
B2 Bayesian Transformer	Best KKT variant (−21 / −6 / −1.4 % at low / moderate / high); confirms the gap is structural
B3 Back-pressure newsvendor	**Beats both strongest baselines in 10/10 scenarios of 3 families (−3 to −25 %)**

Keep the structure, learn only what is uncertain — and let the search pick the structure.

Limitations and future work

What we do not claim

- No real-world validation (3 simulated families)
- Simplified link: budget-enforced pipe, no switch queue, no HARQ, downlink only
- No closed-loop optimality or stability proof
- KKT rule: only −4 % vs proportional in flash crowds
- B2 on Family S only; Bayesian layer too slow in Python for a 10 ms interval
- BPN learned per family, at $T = 10$ ms, $\tau = 1$ ms

Next

- Measured per-cell PRB traces
- PPO xApp baseline; BPN sweeps over T, τ, load
- Cross-family transfer of the learned policy
- Compiled O-DU implementation (C++ / batched)
- Coupled calibration (adjust λ)
- Compression width as a per-cell knob inside the allocation rule (Part A + B)

Thank you — questions?